

SSLA-RT Engineering Handbook

Real-Time Event-Camera Inference
on Jetson Orin NX

A consolidated technical record of the design decisions, optimizations,
failed experiments, and integration work performed during the
porting and tuning of SSLA-S to the Orin NX + DVXplorer Micro target.

2026-05-14

ssla-rt repository — docs/handbook/

Contents

1	Scope and Purpose	3
2	System at a Glance	3
2.1	Reference platform	3
2.2	Pipeline shape	3
2.3	Production performance ceiling	4
3	Hardware-Imposed Invariants	4
3.1	Tegra coherence: pinned memory for cross-CPU/GPU traffic	4
3.2	Halo = 2 at every CPU sharding boundary — LOCKED	5
3.3	Cell-owner warp partition: race-free without GPU atomics	5
4	CPU Pipeline	5
4.1	Topology	5
4.2	NEON optimization rounds	6
4.3	Per-event stage breakdown	7
5	GPU Kernel: <code>ssla_s2_s3_head_celled.cuh</code>	7
5.1	Per-block run loop	7
5.2	Implicit-dispatch layer step	7
5.3	Optimizations applied to the kernel	8
5.4	Smem budget at the production layout	9
6	Real-Time Demo Pipeline	9
6.1	Camera reader thread	9
6.2	Display thread	10
7	Trained-Checkpoint Loading	10
7.1	Checkpoint inspection	11
7.2	Layer index remapping	11
7.3	2-layer YOLOX head structure	11
7.4	Folding BatchNorm into the conv weights	12
7.5	GPU kernel changes for the 2-layer head	12
7.6	Bug: <code>replace_all</code> and duplicated kernel sections	13
7.7	Resolution-mismatch effect on the trained head	13
8	Failed Experiments (Do Not Repeat)	13
8.1	TF32 Tensor Core for L4 qvg projection	14
8.2	Batch-local hidden-cell coalescing	14

8.3	Raising GPU occupancy to 2 blocks/SM	14
8.4	Single-linear head as a fused approximation of the trained head	14
9	Methodology: How To Optimize Without Wasting Time	15
9.1	P1 oracle is gate 0	15
9.2	Trust experiments, not projections	15
9.3	One change per commit	15
9.4	Save dead-end results	16
10	Common Pitfalls and Footguns	16
10.1	NVRTC PTX cache: 80–115 s rebuild after kernel edit	16
10.2	<code>__launch_bounds__(288, 1)</code> is mandatory for the celled kernel	16
10.3	<code>/tmp/ssl_a_s_*</code> is volatile across reboots	16
10.4	<code>kept</code> column includes pad duplicates in P1 bench	16
10.5	<code>cv2.dnn.NMSBoxes</code> asserts <code>score_threshold ≥ 0</code>	16
10.6	DVXplorer Micro contrast threshold valid range is 0–17	16
11	Quick Reference	17
11.1	Build	17
11.2	Generate stub weights	17
11.3	Convert a trained checkpoint	17
11.4	P1 correctness gate	17
11.5	Live run, synthetic source	17
11.6	Live run, DVXplorer Micro, trained checkpoint	17
11.7	Repository layout (production-relevant only)	18
12	Closing Notes	18

1 Scope and Purpose

This handbook is a single-document reference covering the engineering work that took the SSLA-S event-camera detector from a research codebase to a real-time inference runtime on the NVIDIA Jetson Orin NX edge platform. It is intended to be readable cold by an engineer who must:

- resume the optimization work after a break,
- port the runtime to a different Jetson SKU or future GPU,
- swap in a different event camera,
- adapt the design to a different SSLA backbone variant.

It complements three companion documents that already live in the repository:

- `docs/PIPELINE_DESIGN.md` — the canonical pipeline diagram and constraint set.
- `docs/PROFILE.md` — the authoritative latency and throughput numbers (refreshed measurement-by-measurement).
- `docs/DEV_GUIDE.md` — the methodological playbook, hardware-adaptation checklists, and the running record of dead-end optimization attempts.

The handbook does not duplicate those files. Where they go into file-and-line detail, this document presents the consolidated *narrative* of how the system was built and why it looks the way it does, so that future readers can recover not only the answer but also the chain of reasoning that produced it.

2 System at a Glance

2.1 Reference platform

Compute	NVIDIA Jetson Orin NX (8 SMs, sm_87, SM clock pinned at 918 MHz in MAXN).
Sensor	iniVation DVXplorer Micro, 640×480 sensor, USB 2.0, accessed via <code>dv-processing 2.0.3+</code> .
Network	SSLA-S: 4-stage recurrent event backbone with a YOLOX-style detection head. 0.5 M parameters, ~ 96 hidden channels at the final stage.
Throughput target	1 Mev/s (events per second), end-to-end.
Latency target	sub-millisecond emit-to-prediction for output-producing events.

2.2 Pipeline shape

The deployment splits SSLA-S across CPU and GPU. The CPU handles the first two backbone stages (layers L0–L3) and the intermediate spatial downsamples and temporal dropouts. Events are then pushed to a per-block pinned-host ring; a persistent GPU kernel consumes that ring, runs stages 3 and 4 (layers L4–L7), and writes per-cell predictions to a second pinned-host buffer.

End-to-end flow
emit (camera/synth)
↓
CPU shard: L0→L1→pool→tdrop _{s2} →L2→L3→pool→tdrop _{s3}
↓ <i>push</i>
Pinned ring (per-block SPSC, 64 KiB records)
↓ <i>pop</i>
GPU persistent kernel: tdrop _{s2} →L4→L5→pool→tdrop _{s3} →L6→L7→head
↓
Pinned predictions buffer (per-cell, version-tagged)

2.3 Production performance ceiling

Metric	Setting	Value
CPU admit ceiling (stable)	8 shards (n=6 sustainable), halo=2, fused NEON layer	1.85–2.0 Mev/s
GPU drain ceiling	8 blocks, persistent kernel, balanced 2W×4H topology	227–230 kev/s
Per-event GPU kernel p50	at saturation	~310 μ s
End-to-end emit→done p50	at saturation (output-producing events only)	~330 μ s

Numbers are the current values at the end of the May 2026 optimization campaign, with stub weights. They are reproduced from `docs/PROFILE.md` §§1–2; see there for sweeps, percentiles, and historical comparison.

3 Hardware-Imposed Invariants

These constraints are not negotiable design choices: they come from the physical characteristics of the Orin NX and the SSLA-S architecture. Violating any of them silently breaks correctness or causes deadlocks.

3.1 Tegra coherence: pinned memory for cross-CPU/GPU traffic

Orin NX reports `CONCURRENT_MANAGED_ACCESS = 0`. Practical consequence: CUDA managed memory is *not safe* to use for data that the CPU writes while the GPU is concurrently reading. The driver runs in the legacy migrate-on-launch mode, in which a managed page must be fully owned by one side at a time.

Every cross-domain buffer therefore lives in **pinned host memory**, allocated via `cuMemHostAlloc`:

- per-block ring buffers (CPU writes, GPU polls),
- ring head and tail atomics,
- the stop flag,
- the per-cell predictions buffer.

Managed memory (`cuMemAllocManaged`) is reserved for GPU-only state:

- hidden state (read and written by GPU only),
- layer and head weights (CPU writes once at init, GPU reads forever),

- tdrop counters (GPU only after init zero),
- GPU timing slots.

The helpers `cuda_util.alloc_pinned` and `cuda_util.alloc_managed` (Python) document which class each buffer should be in. Using the wrong class produces stale reads that look like correctness bugs.

3.2 Halo = 2 at every CPU sharding boundary — LOCKED

The CPU pipeline shards in the spatial domain. Two adjacent shards must overlap by a halo of at least 2 cells at the resolution of the *coarser* of the two pooling steps that bracket the layer in question. For SSLA-S, this works out to **halo = 2 at the s2 grid**, which after the next s1→s2 pool still gives the needed ± 1 at s3 for the 3×3 patch.

Halo = 1 is *not* sufficient because:

- the 3×3 spatial convolution at the current resolution needs ± 1 cell of halo at that resolution, but
- after the next pool, the same convolution at the next resolution needs ± 1 at that resolution, which is ± 2 cells at the current resolution.

This rule is enforced in `lib_stage01_to_gpu.cpp` and must be preserved by every future sharding scheme. It is one of the few explicit *locked* constraints of the design — the project’s durable rule.

3.3 Cell-owner warp partition: race-free without GPU atomics

The production GPU kernel uses 9 warps per CUDA block, indexed `warp_id` $\in [0, 9)$. Each warp owns the hidden-state cells whose 2-D index satisfies

$$(\text{cy} \bmod 3, \text{cx} \bmod 3) = (\lfloor \text{warp_id} / 3 \rfloor, \text{warp_id} \bmod 3).$$

For any event whose 3×3 patch needs to update 9 hidden cells, the 9 cells map to 9 distinct $(\text{cy}\%3, \text{cx}\%3)$ pairs, hence to 9 distinct owning warps. Therefore **any given hidden cell is written by exactly one warp ever**, eliminating the need for `atomicAdd` or any cross-warp synchronization on hidden state.

The kernel uses `__syncthreads()` only between coarse-grained phases (DISPATCH, COMPUTE, GATHER, TDROP), not inside the per-event step. Within a warp’s queue of (event, delta) tasks, the sequential processing uses only `__syncwarp()` and warp shuffles.

If a future kernel restructuring reduces the warp count or changes the ownership function, the race-free property must be re-proven from scratch, and the P1 oracle drift gate (`bench_s2_s3_head_celled.py`) will catch any violation within a few hundred events.

4 CPU Pipeline

4.1 Topology

The CPU side consists of N shard threads (currently $N = 6$ on cores 1–6) each handling a contiguous Y-band of the input grid with a 2-cell overlap to the next shard. Events whose y coordinate falls inside the band are processed; events on the overlap are processed by both

neighbours and reconciled at the GPU side via ownership (each event ends up routed to exactly one GPU block by the same cell-owner function).

Per-shard work for one event:

$$\text{stage_0 } (L_0 + L_1) \rightarrow \text{pool}/2 \rightarrow \text{tdrop}_{s_2} \rightarrow \text{stage_1 } (L_2 + L_3) \rightarrow \text{pool}/2 \rightarrow \text{tdrop}_{s_3}$$

Each stage applies a *layer_pair*: `matvec`→`LRU` `step`→`matvec_accum`→`gather`→`LayerNorm`. With halo, each shard touches roughly $1/N + 2\Delta$ of the grid where Δ is the halo width in cells.

4.2 NEON optimization rounds

The CPU per-event budget went through three optimization rounds, each documented in `docs/PROFILE.md` and `docs/DEV_LOG.md`.

Round 1 (baseline)

GCC autovectorization on the `matvec` and `LRU` loops. Per-shard rate ~ 135 kev/s; system ceiling ~ 0.82 Mev/s at $n = 8$ shards.

Round 2 (`matvec` + `LRU` NEON)

Hand-written NEON specializations of `matvec_ct<12,36>` and `matvec_ct<24,72>` (the `IN=12` and `IN=24` layers) using 4-output ILP, plus a NEON sigmoid approximation for the `LRU` gate. Per-shard rate rose to 216 kev/s; system ceiling 1.30 Mev/s at $n = 7$.

Round 3 (fused interior layer kernel)

A hand-fused inner kernel that holds `qvg` and `qh` in named NEON registers (`float32x4_t` variables) across the `matvec` → `LRU` → `matvec_accum` sequence, instead of spilling them to a `qvg[18]` array between primitives. Drops the spill cost and lets the compiler keep them register-resident across the `LRU` step. Per-shard rate 327 kev/s at $n = 4$; system ceiling **1.85 Mev/s stable at $n = 6$** , with burst ~ 2.0 Mev/s at $n = 8$.

Pitfall worth remembering

The fused kernel only realized its gain when written with *named* `float32x4_t` variables. A version using `float32x4_t qvg[18]` (array of 18 NEON registers) compiles correctly but does not deliver the speedup — the array form pessimizes the register allocator into spilling. The named-variable form is the canonical recipe, recorded in `ssla_kernels.cpp`.

4.3 Per-event stage breakdown

		Round 1	Round 2	Round 3
$L_0 + L_1$ (<code>stage_forward(0)</code>)	per event	3.94 μ s	1.83 μ s	1.05 μs
$L_2 + L_3$ (<code>stage_forward(1)</code>)	per event	12.88 μ s	9.50 μ s	5.34 μs
Per-shard rate	sustained	135 kev/s	216 kev/s	327 kev/s
System ceiling (stable)	best n	0.82 Mev/s ($n = 8$)	1.30 Mev/s ($n = 7$)	1.85 Mev/s ($n = 6$)

5 GPU Kernel: `ssla_s2_s3_head_celled.cuh`

This is the production GPU kernel. It is launched once at startup as a *persistent* kernel and runs until the host sets a stop flag. There is one CUDA block per GPU block in the topology (currently 8), each pinned to one SM by virtue of the grid dimension equaling the SM count.

5.1 Per-block run loop

1. Sleep-poll the ring tail until at least one event is published.
2. Take the next contiguous run of up to `BATCH = 8` events (the batch never spans non-contiguous slots; consumed in order).
3. Snapshot the batch into shared memory (`event_slots`).
4. Run `tdrops2` for the batch, writing per-event `pass2` flags.
5. Run L_4 (`run_layer_celled_implicit`) and L_5 (`run_layer_celled_implicit_split` with the `pass2` output mask).
6. Pool `s2` coordinates to `s3` (right shift by 1).
7. Run `tdrops3` with the `pass2` mask, writing `pass3` flags.
8. Run L_6 and L_7 (same pattern, with `pass2` as enter-mask and `pass3` as output-mask).
9. Run the YOLOX head for events with `is_owner` $\wedge$ `pass2` $\wedge$ `pass3`; write per-cell predictions and increment the per-cell version counter.
10. Record per-event timing in the timing-slot ring.
11. Advance ring tail; update `events_done` counter.

5.2 Implicit-dispatch layer step

For each layer, the kernel computes, in lock-step across the 9 warps:

1. **DISPATCH**: each warp scans the batch and finds events whose 3×3 patch covers a cell it owns. The corresponding (`event`, `delta`) pairs go into a per-warp task queue.
2. **COMPUTE**: each warp processes its queued tasks sequentially: `qvg matvec` $\rightarrow$ `LRU step` $\rightarrow$ `goW matvec` into a per-event contribution slot. Hidden state writes are exclusive to the owning warp.
3. **GATHER**: one warp per event sums the 9 warp contributions plus the residual, applies LayerNorm, and writes the result back into the event's broadcast slot for the next layer.

Cross-warp synchronization is one `__syncthreads()` between the three phases. Inside a phase, only `__syncwarp()` is needed.

5.3 Optimizations applied to the kernel

L4 qvgIn shared-memory cache (121 KB)

The L_4 qvgIn tensor ($9 \text{ patches} \times 3 \text{ projections} \times 48 \times 24$) is exactly 121 KB. Loading it into shared memory once per block at kernel entry, and reading from smem rather than from global memory in the layer-step matvecs, eliminated the L2/DRAM re-fetch on every batch and removed a measurable L2 thrash signature in `nvprof`. Required `__launch_bounds__(288, 1)` and `cuFuncSetAttribute(MAX_DYNAMIC_SHARED_SIZE_BYTES)` since Orin NX sm_87 supports up to ~ 167 KB dynamic smem per block.

Realized gain: + 4% throughput. (Initial projection was 20–30% — see §9 for the cost of trusting projections.)

L7 qvgIn L2-persistence window (perf bench only)

L_7 's qvgIn (972 KB) is too large for smem but fits the L2 working set if pinned via `CU_STREAM_ATTRIBUTE_ACCESS`. This gives +1.7% offline and approximately 0% live. Currently enabled only in the offline perf bench — not worth a kernel-level dependency.

Balanced $2W \times 4H$ topology (corner blocks taller)

The 8-block topology splits the s2 grid into 2 columns and 4 rows. The default equal-rows split leaves the middle blocks bottlenecked because mid-image typically contains the most motion. Balancing the Y splits to 5/3/3/5 cells brought middle blocks down to the same rate as corner blocks (~ 28 kev/s each), realizing + 4% over the equal split.

State-only (VG-only) path for tdrop-dropped events at L5/L7

Events that fail `tdrops2` or `tdrops3` still need their hidden state to advance (the LRU recurrence is unconditional), but their final output (Q matrix and post-LN projection) is unused. The VG-only branch in `process_patch_cell_state_only` skips the Q matvec and the post-LRU `qh` compute. Realized gain: + 3% throughput.

A1 optimization: skip head for dropped events

Events that fail `tdrops3` have their predictions discarded anyway. The head section skips them entirely (no head matvec, no preds write, no version bump). Trivial save but non-negligible at saturation.

5.4 Smem budget at the production layout

Buffer	Bytes
event_slots[BATCH=8] $\times$ (in_feat + meta)	$\sim 5\,400$
contrib[BATCH][9 warps][OUT_MAX=96]	27\,648
per_warp_in_feat_sm[9][96]	3\,456
per_warp_qh_sm[9][96] (<i>reused for head</i>)	3\,456
task_event / task_delta / task_count / pass2 / pass3	~ 700
L4 qvgIn cache ($9 \times 3 \times 48 \times 24$)	124\,416
Total	$\sim 166\,128$ (rounded to 16 B)

6 Real-Time Demo Pipeline

The live runner (`orin/hybrid_runner_multi.py`) wraps the production kernel with a DVXplorer camera reader thread, a display thread (cv2 window), and a stats loop. This section describes the demo's structure and the pitfalls encountered building it.

6.1 Camera reader thread

```
class CameraReader(threading.Thread):
    SS_LADDER = {"EVERY_PIXEL": 1, "EVERY_SECOND": 2,
                 "EVERY_FOURTH": 4, "EVERY_EIGHTH": 8}

    def run(self):
        import dv_processing as dv
        cam = dv.io.camera.open()
        cam.setSubSampleHorizontal(SS_VAL); cam.setSubSampleVertical(SS_VAL)
        cam.setContrastThresholdOn(contrast)
        cam.setContrastThresholdOff(contrast)
        cam.setCropArea((roi_x, roi_y, roi_w, roi_h))

        while not self.stop_event.is_set():
            batch = cam.getNextEventBatch()
            ev = batch.numpy()
            packed = np.empty((ev.size, 4), dtype=np.float32)
            packed[:, 0] = (ev["timestamp"] - t0).astype(np.float32)
            # Translate to ROI origin then subsample.
            packed[:, 1] = ((ev["x"] - roi_x) // ds).astype(np.float32)
            packed[:, 2] = ((ev["y"] - roi_y) // ds).astype(np.float32)
            packed[:, 3] = ev["polarity"].astype(np.float32)
            self.api.submit(packed[valid])
```

Listing 1: Camera reader, simplified.

Three subtle details:

Library choice. iniVation's older C library `libcaer` (3.3.17) cannot open the DVXplorer *Micro* variant; only the C++ `dv-processing` library $\geq 2.0.3$ does. The Python bindings of `dv-processing` are the easiest path on Orin.

Contrast threshold range. The DVXplorer Micro chip silently clamps the contrast threshold at 17. The `getContrastThreshold()` getter returns whatever value was set, regardless of the clamp, so readback does *not* reveal that values above 17 are ineffective. The full valid range is

therefore 0–17, not 0–255 as the API docs suggest. Use 6–10 for balanced sensitivity, 2–4 for high sensitivity, 12–17 for low.

ROI translation. `setCropArea` crops the sensor in-hardware, but the event coordinates still arrive in sensor-space. Subtract the crop origin before feeding to the SSLA grid, otherwise events miss the grid.

6.2 Display thread

Render at native resolution, scale once

Naively rendering events at the upscaled display resolution and splatting $\text{scale} \times \text{scale}$ squares per event costs $O(\text{scale}^2)$ per event and dominates frame time at 30 fps. The correct pattern is:

1. Build a small canvas of size $H \times W$ (native event resolution).
2. Splat events at native resolution (one pixel per event).
3. `cv2.resize(small, (W×s, H×s), INTER_NEAREST)` for the upscale.

`cv2.resize` is heavily optimized C, while a Python-level splat loop pays the GIL repeatedly. Measured difference: from "feels laggy" to smooth 30 fps.

Frame pacing via `cv2.waitKey`

Use `cv2.waitKey(wait_ms)` for both the GUI event pump and frame pacing, not a Python `time.sleep` loop. The `waitKey` call pumps the X11/Wayland event loop, which a sleep does not, and is also the only portable way to detect the ESC key. Compute the wait budget as $\max(1, \text{period}_{\text{ms}} - \text{elapsed}_{\text{ms}})$.

Non-maximum suppression with `cv2.dnn.NMSBoxes`

The per-cell head produces one bounding box per s3 cell. To collapse duplicates we run NMS across all blocks' candidates:

```
keep_idx = cv2.dnn.NMSBoxes(boxes_xywh, scores,
                             score_threshold=obj_thresh,
                             nms_threshold=nms_iou)
```

Footgun: `cv2.dnn.NMSBoxes` asserts `score_threshold ≥ 0`. If the threshold is computed dynamically (e.g. to accommodate a top-K fallback path that includes sub-threshold candidates), clamp it to zero: `eff_conf = max(0.0, ...)`.

7 Trained-Checkpoint Loading

The May 2026 work added the ability to run the deploy pipeline with a real DSEC-trained SSLA-S checkpoint, not just random/stub weights. This required both Python-side wiring and one structural change to the GPU kernel (a real 2-layer YOLOX head). The section is detailed because the integration touched several seemingly unrelated parts of the code at once.

7.1 Checkpoint inspection

The upstream checkpoint is a PyTorch Lightning `.ckpt` produced by the openeva research repository. The relevant parts:

```
ck = torch.load("best_epoch=41-step=52500.ckpt", map_location="cpu")
sd = ck["state_dict"]           # live optimizer state
ema = ck["ema_state_dict"]      # EMA-of-weights (decay 0.9999), eval-
    preferred
hp = ck["hyper_parameters"]     # dataset (DSEC), height/width, stages, ...
```

Listing 2: Inspecting the checkpoint.

Use the EMA state dict at inference; the live optimizer state is generally slightly worse-generalizing. Both have identical structure for this model.

The state dict contains 96 tensors in two groups: backbone layers (the 8 SSLA-S layers) and a YOLOX-style detection head.

7.2 Layer index remapping

The upstream model uses non-contiguous layer indices because the `spatial_downsample` and `temporal_dropout` operators also take index slots in its `nn.Sequential`-like layout. The deploy uses consecutive indices 0–7. The conversion table is:

Upstream index	Description	Deploy index
0	stage 0, layer 0 (K=1, 2 → 12)	0
1	stage 0, layer 1 (K=3, 12 → 12)	1
4	stage 1, layer 0 (K=3, 12 → 24)	2
5	stage 1, layer 1 (K=3, 24 → 24)	3
8	stage 2, layer 0 (K=3, 24 → 48)	4
9	stage 2, layer 1 (K=3, 48 → 48)	5
12	stage 3, layer 0 (K=3, 48 → 96)	6
13	stage 3, layer 1 (K=3, 96 → 96)	7

Indices 2,3,6,7,10,11 are stateless ops (pool, tdrop) with no parameters and so are not exported.

The conversion script `tools/convert_dsec_ckpt.py` performs this remap, drops unused buffers (`pixel2patch_lut`, `num_batches_tracked`), and writes the standard `weights.npz` + `meta.json` pair that `libstage01_to_gpu` expects.

7.3 2-layer YOLOX head structure

The trained head is markedly more complex than the random single-linear head used during early stub-only testing:

Tensor	Shape	Role
head/cls_convs/0/0/weight	(96, 96, 1, 1)	1×1 conv, classification branch
head/cls_convs/0/0/bias	(96,)	conv bias (absent in stub schema)
head/cls_convs/0/1/*	(96,)×4	BatchNorm ($\gamma, \beta, \mu, \sigma^2$)
head/cls_preds/0/weight	(2, 96, 1, 1)	2 class logits per cell
head/cls_preds/0/bias	(2,)	
head/reg_convs/0/*	(96, 96, ...)	1×1 conv, regression/obj branch
head/reg_preds/0/weight	(4, 96, 1, 1)	4 bbox parameters (tx, ty, log_w, log_h)
head/reg_preds/0/bias	(4,)	
head/obj_preds/0/weight	(1, 96, 1, 1)	objectness logit
head/obj_preds/0/bias	(1,)	

Per-cell forward pass:

$$\begin{aligned}
y_{\text{cls}} &= \text{SiLU}(W_{\text{cls_conv}} \cdot x + b_{\text{cls_conv}}) \\
\text{cls_logits} &= W_{\text{cls_pred}} \cdot y_{\text{cls}} + b_{\text{cls_pred}} \\
y_{\text{reg}} &= \text{SiLU}(W_{\text{reg_conv}} \cdot x + b_{\text{reg_conv}}) \\
\text{reg_out} &= W_{\text{reg_pred}} \cdot y_{\text{reg}} + b_{\text{reg_pred}} \\
\text{obj_logit} &= W_{\text{obj_pred}} \cdot y_{\text{reg}} + b_{\text{obj_pred}}
\end{aligned}$$

The two branches share the input x (the L7 output, $C_3 = 96$ channels) but have *separate* 96→96 conv layers. The non-linearity SiLU prevents fusing into a single matrix.

7.4 Folding BatchNorm into the conv weights

BatchNorm at eval mode is a per-channel affine transform:

$$y_{\text{bn}}[c] = \gamma[c] \cdot \frac{y[c] - \mu[c]}{\sqrt{\sigma^2[c] + \epsilon}} + \beta[c] = \text{scale}[c] \cdot y[c] + \text{shift}[c]$$

with $\text{scale} = \gamma / \sqrt{\sigma^2 + \epsilon}$ and $\text{shift} = \beta - \text{scale} \cdot \mu$. Composing with the preceding 1×1 conv:

$$W_{\text{c}_{\text{out}}, \text{c}_{\text{in}}}^{\text{fused}} = \text{scale}[\text{c}_{\text{out}}] \cdot W_{\text{c}_{\text{out}}, \text{c}_{\text{in}}}^{\text{conv}}, \quad b_{\text{c}_{\text{out}}}^{\text{fused}} = \text{scale}[\text{c}_{\text{out}}] \cdot b_{\text{c}_{\text{out}}}^{\text{conv}} + \text{shift}[\text{c}_{\text{out}}].$$

The folded conv reads identical numbers at inference but saves the explicit BN normalization at every event. Implemented in `hybrid_common.py::_fuse_conv_bn`.

7.5 GPU kernel changes for the 2-layer head

The original kernel had a single linear head: `preds = head_W · L7_out + head_b` with `HEAD_OUT = 7`. The trained head is structurally different, not a fusable approximation.

Added to the kernel:

1. Ten new pointer fields on `HybridS2S3Config`: `head_{cls,reg}_conv_{W,b}`, `head_{cls,reg,obj}_pred`.
2. A `__device__` helper `do_two_layer_head` that runs the per-event 96→96, SiLU, 96→{2,4,1} sequence using one warp per event. The 96-float intermediate is stored in the existing per-warp `qh` smem slot, which is unused after the layer pipeline finishes.
3. A dispatch in the kernel head section: `cfg.head_cls_conv_W != 0` selects the trained path; otherwise the legacy single-linear path runs (used for stub weights).

The Python mirror of the config struct (`hybrid_common.py::CHybridS2S3Config`) gained matching `c_uint64` fields so that the `memmove(p_cfg, addressof(cfg), sizeof(cfg))` call drops the right pointers into the right slots.

7.6 Bug: `replace_all` and duplicated kernel sections

The kernel has *two* structurally identical head sections, one in `k_ssla_s2s3_celled_persistent` (single-block legacy) and one in `k_ssla_s2s3_celled_persistent_multi` (the production multi-block kernel that the live runner actually launches). The two sections differ slightly only in their leading comment line.

When the head dispatch was originally added via a search-and-replace `replace_all=true`, only the first occurrence was matched and updated. The second — the production one — continued to use the old single-linear path that triggers only on `cfg.head_W != 0`. When the Python side wired up the trained head, it set `cfg.head_W = 0` (legacy field unused) and `cfg.head_cls_conv_W = <trained ptr>`. The kernel still ran the old branch, saw `head_W == 0`, and skipped the head entirely. The pinned predictions buffer was therefore never written. On the host side, `sigmoid(0) = 0.5`, hence the symptom: every displayed object score was 0.5.

The diagnosis used a kernel-side `printf` of all 10 head pointers, which confirmed that the pointers *were* reaching the GPU correctly (struct offsets matched between Python ctypes and the CUDA struct). That left "code path not taken" as the only remaining hypothesis, leading to the second head section.

Generalizable lesson: when an apparent search-and-replace covers multiple call sites of the same logic, *open the second site and confirm* that the text matched. Identical-by-intent does not imply identical-by-character. A trailing comment, a missing newline, or different surrounding whitespace silently halves the replacement.

7.7 Resolution-mismatch effect on the trained head

The DSEC checkpoint was trained at 640×430 , stride 8, giving an s3 grid of approximately 80×54 . Initial deploy testing ran at 160×120 (25% linear ROI on the DVXplorer’s 640×480 sensor), giving an s3 grid of 20×15 . The architecture itself is fully spatially translation-equivariant, but the head’s `log_w`, `log_h` regression was trained on absolute pixel scales that no longer match: real objects at 25% ROI appear $4\times$ smaller than the trained scale.

Empirically:

Run-time input	max obj_sigmoid	p99 obj_sigmoid	obj_logit range
160×120 (25% ROI)	0.015	0.011	$[-17, -4]$
640×480 (native, no ROI)	0.95	0.50	$[-18, +3]$

The model is essentially unable to make a confident detection at 160×120 — the head was never trained for that scale.

Rule: for visual verification of a trained checkpoint, run at the trained input resolution. Re-cropping to a smaller window is a training-distribution change as severe as swapping the camera.

8 Failed Experiments (Do Not Repeat)

The most valuable artifact of a careful optimization campaign is the list of attempts that didn’t work. Each entry below was measured end-to-end and reverted.

8.1 TF32 Tensor Core for L4 qvg projection

Hypothesis: replace the per-event L4 qvg matvec ($C1 = 24 \rightarrow 3 \cdot C2 = 144$) with TF32 `mma.sync.m16n8k4`, exploiting Ampere TC.

Implementation: a prototype that pre-packs the qvg matrix into the m=16 format and dispatches the `mma.sync` instruction across a warp.

Result: **P1 unchanged** ($\max|\Delta| = 4.20$, TF32 precision is not a bottleneck for this network). **-5% offline throughput, -3% live.**

Root cause: the m=16 packing required dropping the 121 KB qvgIn smem cache to fit the TC operand buffer. The smem cache was worth more than the TC instruction.

Do not retry without first eliminating the smem cache dependency (e.g. by re-pinning the cache in L2 instead of smem — itself unverified).

8.2 Batch-local hidden-cell coalescing

Hypothesis: within a batch, group the (event, delta) tasks by target hidden cell, then load/store `h_cell` once per group instead of once per task. The motivating observation was that nearby events often hit the same cell.

Implementation: in the layer step, sort the task queue by cell id and fold repeats.

Result: **P1 unchanged. -6% throughput.**

Root cause: synthetic event distributions have reuse factor 1.097 (barely above 1). The bookkeeping overhead (sort + coalesce) was 700× the saving. The script `orin/analyze_coalescing.py` is retained for re-running the analysis on real DVS camera streams where the reuse factor might differ, but the gating heuristic must be very high before this becomes worth the overhead.

8.3 Raising GPU occupancy to 2 blocks/SM

Hypothesis: doubling the number of CUDA blocks per SM (from 1 to 2) would hide memory latency and roughly double throughput.

Measurement: 16-block layout at 96 registers/thread $\rightarrow$ 248 kev/s vs 8-block baseline 227 kev/s. Gain: $+1.1\times$, **not the projected $+1.9\times$.**

Root cause: the kernel is compute/memory-bound at saturation, not latency-bound. Both blocks fight for the same L2 bandwidth and the same scheduler slots. Pipeline-split variants (separate S2 and S3 kernels, or 4-layer split kernels) hit the same wall.

Do not retry as the headline path to higher throughput. Path past the current 230 kev/s ceiling likely requires:

- a topological reorganization that reduces per-event work (e.g. trading the BATCH=8 cell-owner partition for a BATCH=4 with 2 blocks/SM at lower per-event smem),
- channel reduction in L4–L7 via distillation,
- or moving to a newer Jetson with more SMs.

8.4 Single-linear head as a fused approximation of the trained head

Hypothesis: replace the ($96 \rightarrow 96$ BN $\rightarrow$ SiLU $\rightarrow 96 \rightarrow 7$) trained head with a single linear ($96 \rightarrow 7$) using the BN-fused conv matrix composed with the prediction matrix, treating SiLU as identity.

Result: with the linearization, obj confidences during real motion stayed in the noise floor regardless of input scale — the SiLU non-linearity is structural to the head’s behaviour and cannot be dropped.

Conclusion: trained-weight inference *requires* a real 2-layer head on the GPU. Not approximable.

9 Methodology: How To Optimize Without Wasting Time

The optimization rounds documented above all followed the same discipline. The discipline is the most portable lesson from this campaign.

9.1 P1 oracle is gate 0

Every kernel change must pass the P1 oracle (`bench_s2_s3_head_celled.py`) before any performance number is meaningful. The oracle compares `drain_n` output against a single-thread CPU reference; tolerance is $\max|\Delta| \leq 5.0$ on `s3_feat` and `drift = 0` on drop counts.

If P1 fails, stop benchmarking. A wrong answer at full speed is just noise.

9.2 Trust experiments, not projections

Every optimization estimate in this codebase has been off by 2–10 $\times$, almost always overestimating. The actual gains versus projections:

Optimization	Projection $\rightarrow$ actual
L4 <code>qvgIn</code> smem cache	+20–30% $\rightarrow$ +4%
L7 L2 persistence	+5–10% $\rightarrow$ +1.7% offline, 0% live
Pipeline split / 2 blocks/SM	+50–100% $\rightarrow$ +9% then +1.1 $\times$ at higher occupancy
Balanced topology	+20–30% $\rightarrow$ +4%
VG-only state-only patch	+6–13% $\rightarrow$ +3%
TF32 TC at L4	uncertain (expert promised gain) $\rightarrow$ –5%
Batch-local hidden coalescing	uncertain (expert promised gain) $\rightarrow$ –6%

Rule: for any proposed optimization, write the smallest patch that can deliver the gain, measure it, and only scale up if the measurement is within 2 $\times$ of the projection. Otherwise, debug or abandon.

9.3 One change per commit

Bundling makes regressions hard to bisect. Each commit message records:

- what changed (one sentence),
- why it should help (one paragraph of analysis),
- measured P1 result (drift, $\max|\Delta|$),
- measured perf result (throughput, latency),
- revert decision if applicable.

9.4 Save dead-end results

A negative result is more valuable than a missing measurement. The "failed experiments" section above only exists because the dead ends were written down at the time. The auto-memory entries `project_tf32_tc_l4_prototype`, `project_coalescing_net_negative`, and `project_occupancy_gives_litt` encode them as machine-readable notes that survive across conversations.

10 Common Pitfalls and Footguns

A grab-bag of issues that bit us during this campaign and are worth documenting at the head of any future port.

10.1 NVRTC PTX cache: 80–115 s rebuild after kernel edit

The first compile of a kernel variant takes 80–115 seconds. Subsequent runs hit the cache in ~ 0 s. The cache lives at `~/.cache/openeva-orin-ptx/` (or `~/.cache/openeva/orin_ptx/`) and is keyed by the hash of the kernel source string. Editing the source — even adding a comment — changes the hash and forces a rebuild.

Set `OPENEVA_ORIN_NO_PTX_CACHE=1` to skip the cache entirely (useful when verifying cache correctness or after suspected struct layout changes that didn't trigger an obvious rebuild).

10.2 `__launch_bounds__(288, 1)` is mandatory for the celled kernel

NVRTC's default `MAX_THREADS_PER_BLOCK` is 256. The celled kernel uses $9 \text{ warps} \times 32 = 288$ threads. Without the explicit hint, `cuLaunchKernel` returns `LAUNCH_OUT_OF_RESOURCES` (error 701).

10.3 `/tmp/ssla_s_*` is volatile across reboots

Orin clears `/tmp` on reboot. If `hybrid_runner` fails with "cannot open `/tmp/ssla_s_*/meta.json`", regenerate via `tools/make_ssla_stub.py` (stub) or `tools/convert_dsec_ckpt.py` (trained).

10.4 `kept` column includes pad duplicates in P1 bench

When the P1 oracle pads block ring sizes to a multiple of `BATCH = 8`, the duplicates pass through the kernel and inflate the `kept` count. Dedupe via global event index when comparing to the CPU reference.

10.5 `cv2.dnn.NMSBoxes` asserts `score_threshold ≥ 0`

Already covered in §6.2. The dynamic threshold for top-K fallback can go slightly negative; clamp.

10.6 DVXplorer Micro contrast threshold valid range is 0–17

Already covered in §6.1. The getter does not reveal the silent clamp.

11 Quick Reference

11.1 Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

Produces three shared libraries: `libstage01_to_gpu.so` (the production hybrid pipeline), `libstage01_capi.so` (CPU-only baseline through L_3), `libstage0_capi.so` (CPU-only through L_1).

11.2 Generate stub weights

```
python3 tools/make_ssla_stub.py /tmp/ssl_a_s_64x80/ --h 64 --w 80
```

11.3 Convert a trained checkpoint

```
python3 tools/convert_dsec_ckpt.py \
/path/to/best_epoch=N-step=M.ckpt \
/tmp/ssl_a_s_dsec_480x640/ --h 480 --w 640
```

11.4 P1 correctness gate

```
cd orin
python3 bench_s2_s3_head_celled.py --n 200000
# expect: drift = 0, max|delta| <= 5
```

11.5 Live run, synthetic source

```
cd orin
python3 hybrid_runner_multi.py \
--weights /tmp/ssl_a_s_64x80/ --h-full 64 --w-full 80 \
--gpu-blocks 8 --shards 6 \
--cpp-synth --synthetic-mev 2.0 --duration-s 8
```

11.6 Live run, DVXplorer Micro, trained checkpoint

```
cd orin
python3 hybrid_runner_multi.py \
--weights /tmp/ssl_a_s_dsec_480x640/ --h-full 480 --w-full 640 \
--gpu-blocks 8 --shards 6 \
--roi-pct 100 --contrast 17 --cam-subsample EVERY_PIXEL \
--demo-display --display-fps 15 --display-scale 1 \
--obj-thresh 0.3 --nms-iou 0.65 \
--duration-s 60
```

11.7 Repository layout (production-relevant only)

```
ssla-rt/
  include/                public C/C++ headers
  src/lib_stage01_to_gpu.cpp  production hybrid pipeline (CPU s0+s1 -> ring
    -> GPU)
  src/lib_stage01_capi.cpp    CPU-only stages 0+1
  src/ssla_kernels.cpp        per-stage SSLA-S forward kernels (NEON-
    optimized)
  tools/make_ssla_stub.py     random-weight stub generator
  tools/convert_dsec_ckpt.py  DSEC ckpt -> deploy schema conversion
  orin/orin/hybrid_common.py  ctypes struct mirrors, helpers
  orin/orin/multi_block.py    per-block resource allocation + topology
  orin/orin/weights_ssla.py   reshape ssla weights into matvec-friendly
    layout
  orin/kernels/ssla_s2_s3_head_celled.cuh  PRODUCTION GPU kernel
  orin/kernels/proto_layer_pair.cuh        warp-cooperative primitives
  orin/hybrid_runner_multi.py  N-block production live runner
  orin/bench_s2_s3_head_celled.py P1 oracle harness
  orin/perf_celled_multi.py    offline throughput
  docs/PIPELINE_DESIGN.md      canonical pipeline + constraints
  docs/PROFILE.md              current measurements
  docs/DEV_GUIDE.md            methodology playbook
```

12 Closing Notes

The system in its current state delivers ~ 230 kev/s GPU drain and $\sim 330 \mu\text{s}$ p50 end-to-end latency at saturation on Jetson Orin NX with stub weights, and runs the trained DSEC checkpoint end-to-end through the production kernel via a real 2-layer YOLOX head on the GPU.

The remaining headroom is not on the CPU — which already produces 1.85–2.0 Mev/s, well above the 230 kev/s GPU ceiling — but on the GPU side. The paths considered for breaking past 230 kev/s (higher occupancy, pipeline splitting, TF32 TC) all gave $\leq 1.1\times$ in measurement. A more invasive change (channel reduction via knowledge distillation, or a different topology that trades BATCH for blocks per SM) is the most plausible next direction. None of those are shipped.

For visual verification of a trained checkpoint, always run at the trained input resolution; the head's `log_w`, `log_h` regression does not transfer across resolution.

Everything in this handbook should be reproducible from the repo at the May 2026 head-of-line commit. The authoritative numbers live in `docs/PROFILE.md`, refreshed every measurement-bearing optimization commit.